mojo-adreno · Adreno GPU vs Hexagon NPU

GPU or NPU? What we measured on Snapdragon

Mojo kernels on the Adreno 840 and X2-90 GPUs against fp16 graphs on the Hexagon NPU (HTP V81), through ONNX Runtime and QNN: a mixture of experts, Llama's and π0's prefill, and Gemma 4 on the phone's GPU.

4.1×
Llama 3.2 1B prefill on the NPU: 168 ms against 685 ms on the GPU

2.9×
π0's Gemma 2B prefix on the NPU: 0.80 s against 2.34–2.35 s

16×
the work a static NPU graph does for an MoE: all 128 experts, not 8. The GPU keeps it.

28.1 ms
a Gemma 4 E2B token in int4 on the Adreno 840; int8 closes the gap to bf16

Galaxy S26 Ultra, Galaxy Z Fold SM-F976U (Samsung Remote Test Lab) · Snapdragon X2 Elite laptop (Qualcomm Device Cloud)

Two ways onto the chip

GPU: Adreno 840, Adreno X2-90

- Mojo `DeviceContext` kernels, SPIR-V from the fork's adreno840 target, through Vulkan.
- Decided at run time: a kernel reads the router's expert ids; a prompt of any length.
- int4 and int8 weights, unpacked inside the kernels.
- Prefill on KHR cooperative matrices (fp32, 64 × 64 × 8).

NPU: Hexagon HTP V81

- fp16 ONNX graphs through ONNX Runtime's QNN execution provider (ORT 1.29 + QNN 2.42 from Maven Central).
- Compiled once, on the device, for fixed shapes: 448 rows for Llama (201 s), 832 for π0 (18.5 min).
- Compiled contexts load at each process start: 1.9–2.4 s (Llama).
- Quantized weights only per output channel (w8a16, w4a16).

NPU prefill → **K/V cache + first token** → **GPU decode**

Each part's K/V is copied while the NPU runs the next: 0.8–2.0 ms left after the last part (Llama).

Sources: npu/README.md, pi0/README.md · the X2 Elite laptop uses ONNX Runtime's QNN NuGet package 1.24.4 for Windows on Arm

An MoE: dynamic on the GPU, static on the NPU

Qwen3-30B-A3B, int4: 48 layers, each with 128 experts; a router picks 8 per token, so a token uses 3.3B of 30.5B parameters. Snapdragon X2 Elite laptop.

GPU: reads only the chosen experts

1. **Router**: top 8 of 128; ids and weights stay in GPU memory.
2. **Decode**: expert kernels find their int4 rows by those ids. 1.63 GB of weights a token.
3. **Prefill**: tokens grouped by expert (about 25 each of 404); each expert's weights read once, a grouped GEMM.

GPU prefill	Experts grouped	A token at a time
128 tokens	**1.66–1.71 s**	4.02–4.25 s
256 tokens	**2.57–2.66 s**	7.48–7.80 s
404 tokens	**4.24–4.31 s**	11.8–13.1 s

NPU: a static graph computes them all

1. **No gather**: a QNN graph cannot select weights by an index computed inside it.
2. **TopK as a dense weight** per token: 8 non-zero of 128.
3. **All 128 experts** run for every token, weighted: 16× the useful arithmetic.

And it does not fit as fp16

The whole model is 61 GB in fp16, against the laptop's 33.6 GB. On the HTP it would need the quantized path (w4a16) and contexts of a few layers each.

Source: qwen3moe/README.md ("How a token runs", "Prefill", "On the Hexagon NPU") · ranges over two laptop sessions

Qwen3-30B-A3B: the GPU wins both phases

Qwen3-30B-A3B	NPU: one layer, fp16	NPU: 48 × that layer	GPU: int4, all 48 layers
Decode, a token	10.7–14 ms	0.5–0.7 s	**27–32 ms** (31–37 tokens/s)
Prefill, 128 tokens	213–218 ms	10.3 s	**1.66–1.71 s**
Prefill, 256 tokens	95–97 ms	4.6 s	**2.57–2.66 s**

The hardware is fast

The HTP streams fp16 weights at about 100 GB/s, against 55–60 GB/s for our GPU kernels, and runs 3.4 TFLOP/s of fp16 matrix work.

The graph is the limit

A decode token moves all 128 experts' 1.2 GB a layer. Reading only 8 would be about 0.8 ms a layer (README's estimate).

Accurate in fp16

Median token within 1.2e-3 of transformers' fp32 layer. QK norms written as L2 norms: QNN's RmsNorm refuses heads of 128.

Shape-dependent plans

QNN compiled the 128-token graph to a slower plan than the 256-token one: 213–218 against 95–97 ms.

So for this MoE the GPU decodes and prefills. The NPU could take the dense attention, or a prefill with expert gathering, once QNN graphs can select weights by index.

Source: qwen3moe/README.md ("On the Hexagon NPU") · layer 0, real weights · "48 × that layer" is the README's scaling, not a run

Llama 3.2 1B: the NPU prefills 4.1× faster, decode ties

Llama 3.2 1B, 431-token prompt	Prompt: prefill + handoff	Decode a token	Tokens vs PyTorch
All on the NPU, fp16	**169 + 0.4 ms**	38.8 ms · 25.7 tok/s	80 of 80
NPU prefill + GPU decode, fp16	**169 + 1.4 ms**	40.0 ms · 25.0 tok/s	80 of 80
All on the GPU, fp16	685 ms	39.8 ms · 25.2 tok/s	80 of 80
NPU prefill (fp16) + GPU decode, 4-bit	**169 + 0.8 ms**	**13.2 ms · 76.0 tok/s**	38 of 80 (a near-tie); 80 of 80 from the NPU's K/V
All on the GPU, 4-bit	641 ms	**13.2 ms · 75.7 tok/s**	80 of 80 (int4 throughout)

Prefill is matrix work

2,550 tokens/s on the HTP against 629 (fp16) and 672 (4-bit) on the Adreno: 4.1× and 3.8×.

Decode is memory

Each token reads 2.47 GB of fp16 weights; both run near the same memory speed, about 63 GB/s.

4-bit decode: the GPU

13.2 ms, 3× fp16. The NPU has no 4-bit row: int4 on the HTP needs quantized activations (w4a16).

fp16 on the HTP is exact enough

K/V within 2.2e-3 relative, logits within 0.027 of PyTorch: closer than ONNX Runtime's fp16 CPU kernels.

Source: npu/README.md · Galaxy S26 Ultra SM-S948U, warm, SKIN below 40 °C · 4-bit row: 38 of 80 against a mixed fp16/int4 reference

Quantized weights on the NPU: per channel, not blocks

Llama 3.2 1B on the HTP, 431 tokens	Prefill	Contexts	K/V error vs the fp16 model	Tokens vs PyTorch (fp16)
fp16	168 ms	2.5 GB	2.2e-3	80 of 80
w8a16: int8 per channel	**116 ms**	1.29 GB	3.1e-2	29 of 80, then a paraphrase
w4a16: int4 per channel	**111 ms**	0.68 GB	0.60	14 of 79
w4a16: int4 blocks of 64 (LPBQ)	495 ms	0.72 GB	0.35	16 of 71; 71 of 71 vs the int4 model

Ternary Qwen3-8B, 408 tokens, w4a16 per channel

	Prompt	Decode	Tokens
NPU prefill + GPU decode	**1.64 s** = 1.37 + 0.23	48.0 ms	the same 80
All on the GPU	3.60 s	45.4 ms	80

2.6× faster prefill; the int4 weights approximate the ternary group scales (K/V 9.7e-2 vs the exact model) and still give all 80 tokens.

Why blocks lose

ORT 1.29's QNN provider maps block int4 only as LPBQ. On 448 × 2048 × 16384: fp16 4.25 ms, int8 per channel 3.94, int4 per channel 3.75; LPBQ as MatMul 23.4 ms, as FullyConnected a crash except at 2048 × 2048.

Source: npu/README.md on branch npu-quant (7906a53) and its session notes · Llama: Galaxy S26 Ultra · Qwen3-8B: Galaxy Z Fold

π0: Gemma 2B's prefix on the NPU, the rest on the GPU

π0, a 50 × 32 action chunk	Phone: NPU prefix	Phone: all GPU	Laptop: NPU prefix	Laptop: all GPU
Vision, SigLIP (GPU)	494–496 ms	494–514 ms	378–383 ms	378–389 ms
Prefix, Gemma 2B	**0.80 s**	2.34–2.35 s	**0.81 s**	2.13–2.15 s
10 denoising steps (GPU)	745–748 ms	749–784 ms	674–701 ms	0.70–0.74 s
Action chunk	**2.05–2.16 s**	3.59–3.61 s	**1.82–1.85 s**	3.1–3.9 s

Prefix 2.9× and 2.6×

18 layers over 832 rows, 3.2 TFLOP, in 0.79 s on the HTP (4.0 TFLOP/s). The chunk: 1.7× on the phone, 1.7–2.1× on the laptop.

Accurate enough

Actions within 3.2e-3 of PyTorch fp32 (the GPU alone: 1.1e-5). All of it is the fp16 prefix: PyTorch's steps from the handed-off K/V match within 1.4e-6.

Less on the GPU

The GPU holds no Gemma 2B weights: 1.4 GB instead of 5.4 GB. Vision and the 10 steps are now most of the chunk.

Laptop: K/V in place

The Vulkan server runs the NPU and writes K/V into the GPU's mapped buffers: 2.4–3.3 ms beyond the graphs.

Sources: pi0/README.md (the NPU prefix; the laptop's chunk), vla/x2/README.md · phone: Galaxy Z Fold SM-F976U · laptop: its own clock

Where the NPU helps, and where it does not

Measured	NPU (HTP)	GPU (Adreno)	Faster, and why
Llama 3.2 1B prefill, 431 tokens, fp16	**168 ms**	685 ms	**NPU, 4.1×**: dense matrices, fixed shape
Llama 3.2 1B prefill, w8a16 on the NPU	**116 ms**	685 ms (fp16)	**NPU, 5.9×**, but 29 of 80 tokens
Ternary Qwen3-8B prefill, 408 tokens	**1.37 s** (w4a16)	3.60 s	**NPU, 2.6×**, the same 80 tokens
π0 prefix, Gemma 2B (phone)	**0.80 s**	2.34–2.35 s	**NPU, 2.9×**
π0 prefix, Gemma 2B (X2 Elite laptop)	**0.80–0.84 s**	2.13–2.15 s	**NPU, 2.6×**
Llama 3.2 1B decode, fp16	38.8 ms	39.8 ms	**A tie**: both memory-bound, about 63 GB/s
Llama 3.2 1B decode, 4-bit	no graph	**13.2 ms**	**GPU**: int4 needs w4a16 on the HTP
Qwen3-30B-A3B decode (MoE)	0.5–0.7 s*	**27–32 ms**	**GPU**: the graph runs all 128 experts
Qwen3-30B-A3B prefill, 256 tokens	4.6 s*	**2.57–2.66 s**	**GPU**: experts grouped by token
SmolVLA prefix, 241 rows (phone)	not run	62 ms	62 ms of a 776–778 ms chunk

* one fp16 layer on the NPU, times 48 (qwen3moe/README.md's scaling); the whole model does not fit in fp16.

Sources: npu/README.md and branch npu-quant, pi0/README.md, qwen3moe/README.md, smolvla/README.md · devices as on earlier slides

Gemma 4 E2B and E4B on the Adreno 840, int4

Galaxy Z Fold, int4	E2B	E4B
Layers × hidden	35 × 1536	42 × 2560
`gemma4.bin`, on the GPU	1.21 GB	2.47 GB
Decode	**28.1 ms** · 35.6 tok/s	**53.8 ms** · 18.6 tok/s
960-token prompt, batched (vs a token at a time)	**2.36 s** · 406 tok/s (13×)	**4.71 s** · 204 tok/s (15×)
Logits vs transformers, same weights	within 1.7e-4	within 1.7e-4
bf16's greedy choice agrees (short, long)	42 of 47, 49 of 64	39 of 48, 55 of 64

Decode reads every weight

1.21 GB in 28.1 ms (43 GB/s), 2.47 GB in 53.8 ms (46 GB/s); 425 and 509 launches a token.

Prefill on the matrix units

int4 projections on cooperative matrices are 80–86% of a prompt, at 1.7–2.1 TFLOP/s. Attention there: 0.28 s, was 1.70 s scalar.

What is new in Gemma 4

Later layers reuse earlier K/V (the last 20 / 18); per-layer inputs from a 4.7 GB table (E2B) left in the checkpoint, a row a token; sliding window 512.

The gap to bf16 is int4 rounding, not the kernels: the phone matches transformers on its own weights, and the error is spread over every kind of matrix.

Source: gemma4/README.md, branch gemma4 (52c2951, not merged) · Galaxy Z Fold SM-F976U · bf16 teacher-forced on the phone's tokens

int8 closes the gap to bf16, at 1.5× the decode time

Gemma 4 storage	int4	mixed	int8
Matrices in 8 bits	none	all but the head and the K/V-sharing layers' MLPs	all but the head
`gemma4.bin`, E2B / E4B	1.21 / 2.47 GB	1.59 / 3.75 GB	2.15 / 4.46 GB
bf16 agrees, E2B: short · long	42/47 · 49/64	49/55 · 63/64	**53/54 · 64/64**
bf16 agrees, E4B: short · long	39/48 · 55/64	52/54 · 59/64	**52/52 · 64/64**
Mean KL(bf16 ‖ it), E2B	0.30 · 0.62	0.14 · 0.032	**0.012 · 0.0054**
Mean KL(bf16 ‖ it), E4B	0.41 · 0.34	0.038 · 0.022	**0.0031 · 0.0033**
Decode, E2B / E4B	**28.0 / 54.2 ms**	33.1 / 73.0 ms	41.1 / 83.0 ms
960-token prefill, E2B / E4B	2.35 / 4.68 s	2.37 / 4.73 s	2.44 / 4.76 s

8 bits, not finer int4

Simulated on E2B's long prompt: int4 per 32 only takes the KL from 0.62 to 0.45; int8 per 64 to 0.0024.

The head stays int4

It adds only 0.006 of KL in int4, and saves 0.20 GB (E2B) or 0.34 GB (E4B) of every token's reads.

What it costs

Decode reads 1.8× the bytes: 1.47× int4's time (E2B), 1.53× (E4B). Prefill 2–4% more: products, not reads, set it.

Still exact

Every storage is within 1.9e-4 of transformers on its own weights, every greedy token matching.

Source: gemma4/README.md ("int8"), branch gemma4 (52c2951) · short: the sky prompt; long: 960 tokens · times from a second Fold

What decides GPU or NPU

Dense, fixed-shape prefill: the NPU

2.6–4.1× on Llama, ternary Qwen3-8B and π0. Overlapped with the NPU, the K/V handoff leaves 0.8–2.0 ms (Llama), 2.4–3.3 ms (π0, laptop).

Memory-bound decode: a tie in fp16

38.8 against 39.8 ms. With int4 weights the GPU is 3× faster; the NPU would need quantized graphs.

Routing, run-time shapes: the GPU

An MoE's static graph computes all 128 experts; the GPU reads the 8 chosen and groups tokens by expert.

Next

- QNN graphs that gather experts by index, or the NPU taking the MoE's attention.
- Chunked NPU graphs with a K/V input, for any prompt length without recompiling.
- Gemma 4's prefill on the HTP, as for π0's prefix.
- Between int4 and int8 for Gemma 4: 5–6 bits, or GPTQ-style rounding for mixed's int4 MLPs.

github.com/adityaGoCT/mojo-adreno · npu/ · qwen3moe/ · pi0/ · vla/x2/ · gemma4/ on branch gemma4 · npu/ quantized graphs on branch npu-quant